

Unit 2

Data Preprocessing and EDA

Data set: attribute types, explanatory variable, outcome variable

Attribute Types

Nominal:

Categories with no inherent order — labels only.
Example: blood type, zip code, marital status.

Interval

Numeric, ordered, equal intervals, but no true zero. Example: temperature in °C — "20° is twice 10°" isn't valid.

Ordinal

Categories with a meaningful rank, but unequal or undefined gaps between them. Example: education level, satisfaction rating.

Ratio

Numeric, ordered, equal intervals, with a true zero meaning "none." Example: height or income — 10 kg is truly twice 5 kg.

Explanatory Variable(s)

- Also called independent, predictor, or feature variables — the inputs a model uses to explain or predict an outcome. A data set usually has many. Example: predicting a house's sale price from its square footage, location, and age.

Outcome Variable

Also called the dependent, target, or response variable — the quantity or category being explained or predicted. A supervised task has exactly one. Example: the sale price itself, the value the model is built to estimate.

Data Preprocessing

Refers to the techniques and procedures used to prepare raw data into a clean, organized and structured format suitable for analysis or modeling.

- It improves data quality
- Enhances model performance
- Reduces computational complexity
- Ensure compatibility
- **Different ways of data preprocessing**
 - Missing data,
 - Smooth out noise,
 - Handling outliers,
 - categorical encoding,
 - Feature scaling and
 - Normalization, and Discretization.

Measures of Center (Averages)

- **Mean:** Add all numbers together and divide by the total count of values.
- **Median:** Put the numbers in order from least to greatest and find the middle value. If there is an even count, take the average of the two middle numbers.
- **Mode:** The number that shows up the most times in your set of data. A set can have one mode, more than one mode, or no mode at all.

Measures of Spread (Variability)

- **Variance:** Measures how far each number is from the mean. You subtract the mean from each data point, square the result, add them all up, and divide by the count (or count minus one for a sample).
- **Standard Deviation:** The square root of the variance. It puts the measure of spread back into the original units of your data, showing how much values typically vary from the mean

Handling Missing Data

- Ignore the tuple: drop records with missing values — simple, but wastes data and can bias results if missingness isn't random.
- Fill manually: a domain expert supplies the value — accurate, but slow and impractical at scale.
- Fill with a global constant, e.g., "Unknown" or 0 — fast, but risks the model treating the fill as a real category.
- Fill with mean or median (overall or by class) — easy, but understates variability and weakens correlations.
- Predict the value via regression, decision-tree, or k-NN imputation — most accurate, at higher computational cost.

Handling Missing Data

```
import numpy as np
import pandas as pd

# Sample dataset with missing values (NaN)
data = {'Age': [25, np.nan, 30, 45, 22], 'Salary': [50000, 60000, np.nan, 80000, 52000]}
df = pd.DataFrame(data)
df_dropped = df.dropna()
print(df_dropped)

# Fill with a global constant (e.g., 0 for Age)
df_constant = df.fillna({'Age': 0, 'Salary': 45000})
print(df_constant)

from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='mean') # or 'median'
df_mean = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)
print(df_mean)
```

Data noise

Data noise is random variations or anomalies in a dataset that don't represent meaningful information. Often, these are small fluctuations, but even minor noise can disrupt analysis, leading to unreliable insights and patterns that don't hold up under scrutiny.

Types of data noise

- 1. Random variations:** Small differences in how data is collected or processed over time, resulting in minor, unpredictable changes in data that can distort findings Ex. [time series](#) data.
- 2. Measurement errors:** Errors from faulty instruments or inconsistent methodologies, These lead to skewed results and misinterpretations if left unchecked.
- 3. System interference:** Digital systems occasionally experience interference, think of network lag or software bugs, that introduces noise into datasets, especially in real-time data streams.
- 4. Seasonal fluctuations:** Regular, predictable patterns, like monthly sales spikes or annual purchasing cycles, introduce variability.
- 5. Outliers vs. legitimate extreme values:** Outliers can sometimes signal genuine extremes worth analyzing, but distinguishing these from noise requires a critical eye

Impact of on analysis

Unmanaged noise can impact data analysis in several ways:

- **False patterns and correlations:** Noise produces false relationships that don't truly exist, causing you to chase "insights" that lead to misguided decisions.
- **Skewed statistical measures:** Core metrics, like mean and median, can be thrown off by noise, affecting how data is summarized and understood.
- **Misleading visualizations:** When noise isn't addressed, charts might show erratic trends that overshadow the actual data patterns.
- **Reduced model accuracy:** Predictive models trained on noisy data often underperform, as they're trying to account for patterns that don't really exist.
- **Resource inefficiency:** Time and resources are wasted on analyzing false patterns, diluting the focus on meaningful insights.

Smoothing Noisy Data

Binning: partition sorted values into equal-width or equal-frequency bins, then replace each with its bin mean, median, or boundary.

Data binning (or bucketing) converts continuous numerical values into smaller, discrete intervals or categories. Common methods include equal-width binning (dividing the data range into equal intervals) and equal-frequency binning (placing an equal number of data points into each bin)

Equal-Width Binning Example

Divides the total data range (Max - Min) into N intervals of equal size. Formula for width $W = \frac{Max - Min}{N}$

Imagine a dataset of employee ages: [22, 25, 31, 35, 42, 58].

Min value: 22, Max value: 58. Suppose we want 3 bins.

Bin width calculation: $(58 - 22) / 3 = 12$.

Bin ranges:

Bin 1 (22 to 34): contains 22, 25, 31

Bin 2 (34 to 46): contains 35, 42

Bin 3 (46 to 58): contains 58

Equal-Frequency (Equi-Depth) Binning

Divides the sorted data so that each bin contains approximately the same number of data points, regardless of the interval range width.

Example: For the same 8 data points sorted [4, 8, 9, 15, 21, 21, 24, 28] split into 2 bins:

Bin 1: First 4 items [4, 8, 9, 15]

Bin 2: Last 4 items [21, 21, 24, 28]

Bin Smoothing Techniques

Once bins are formed, values can be smoothed to reduce noise:

- **Smoothing by Bin Means:** Replaces each value in a bin with the average value of that bin. For Bin 1 [4, 8, 9, 15], the mean is $(4+8+9+15)/4 = 9$, changing elements to [9, 9, 9, 9].
- **Smoothing by Bin Medians:** Replaces each value with the median value of the bin.
- **Smoothing by Bin Boundaries:** Replaces values with the closest minimum or maximum boundary value of that specific bin.

Regression: fit the data to a function (linear or multiple regression) and use the fitted values in place of the noisy ones.

Regression for smoothing in machine learning uses local regression models, splines, or penalties to clean noisy data and fit smooth non-linear curves.

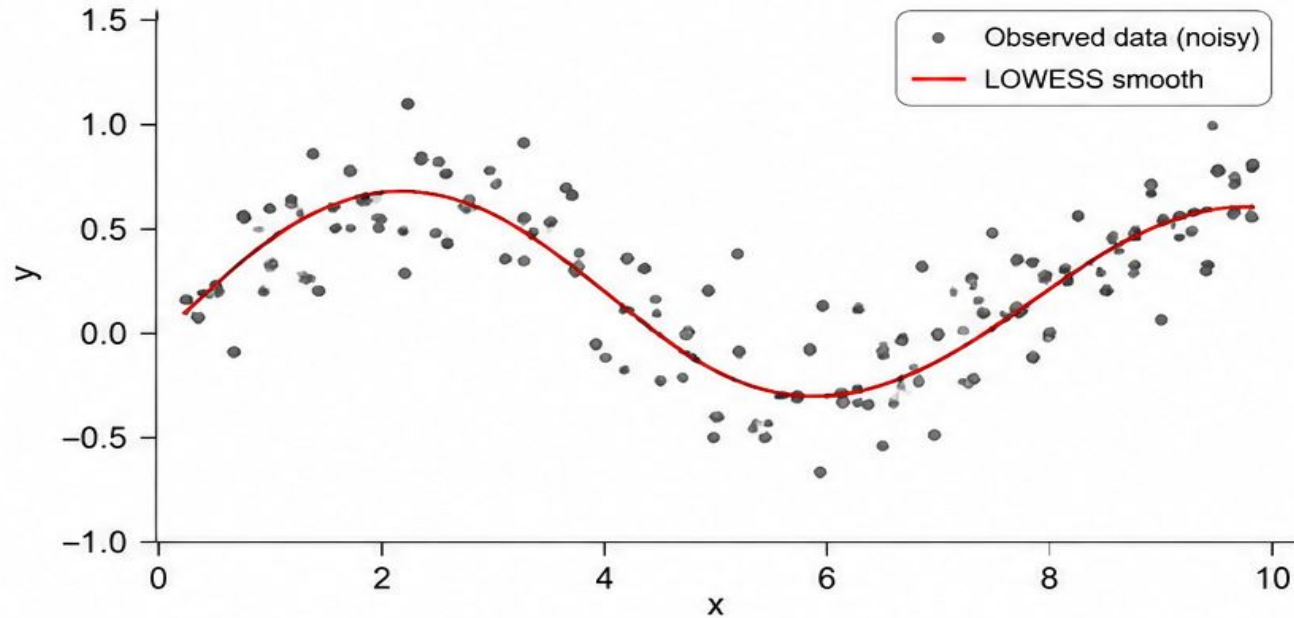
Regression Smoothing Methods

- **LOESS / LOWESS(Locally estimated scatter plot smothing):** Fits simple local regression models inside small moving windows of data points using weighted least squares.
It is a technique used to **draw a smooth curve through scattered data points** without assuming the data follows a straight line or a specific equation.
Example: Imagine you have the following marks of students over time : If you simply join the points, the graph will look zigzag because marks increase and decrease slightly. LOESS creates a smooth trend line that shows the overall pattern.
- **Regression Splines:** Divides the input range into regions separated by knots and fits low-degree polynomials that join smoothly at boundaries.
- **Kernel Smoothing:** Computes a weighted average of neighboring data points where closer points get higher weights based on a kernel function.
- **Regularized Regression (Ridge):** Smooths coefficient estimates by adding a penalty term to reduce variance and prevent overfitting.

LOWESS (Locally Weighted Scatterplot Smoothing)

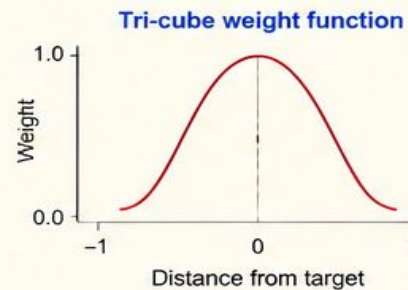
LOWESS fits simple models to localized subsets of the data to build up a curve that describes the underlying trend.

1. Overall View



How LOWESS works (intuition)

1. Choose a target point on the x-axis.
2. Select neighboring points within a window (span).
3. Assign weights: points closer to the target get higher weights (tri-cube weighting).
4. Fit a simple linear regression to these weighted points.
5. The fitted value at the target point is the smoothed value.
6. Move to the next target point and repeat.



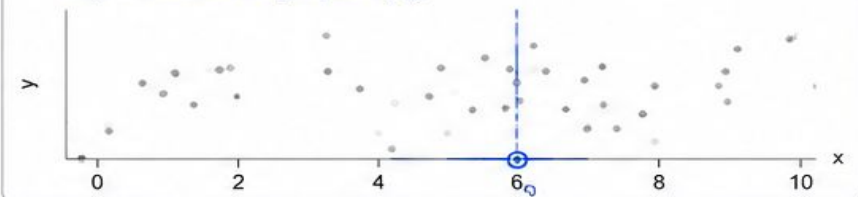
Span (window size) controls the amount of smoothing:

Small span → follows data closely (wiggly)

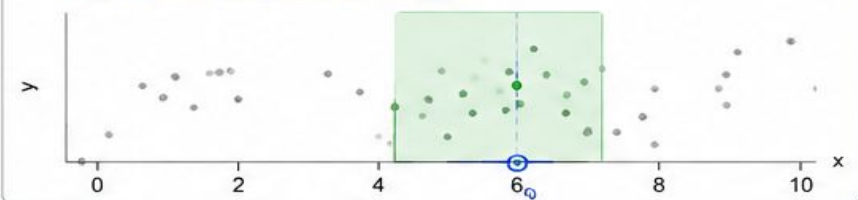
Large span → smoother curve

2. Local Fitting at a Target Point

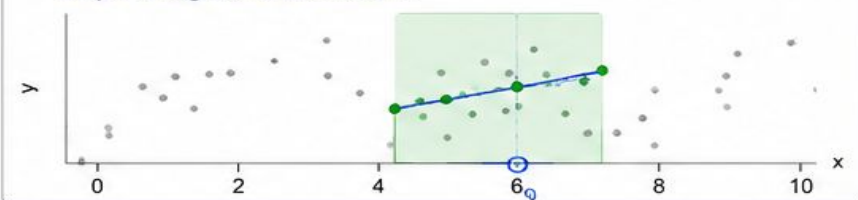
Step 1: Choose target point (x_0)



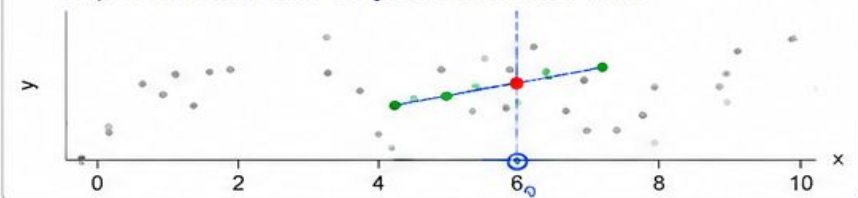
Step 2: Select neighbors within span



Step 3: Weight and fit local line



Step 4: The fitted value at x_0 is the smoothed value



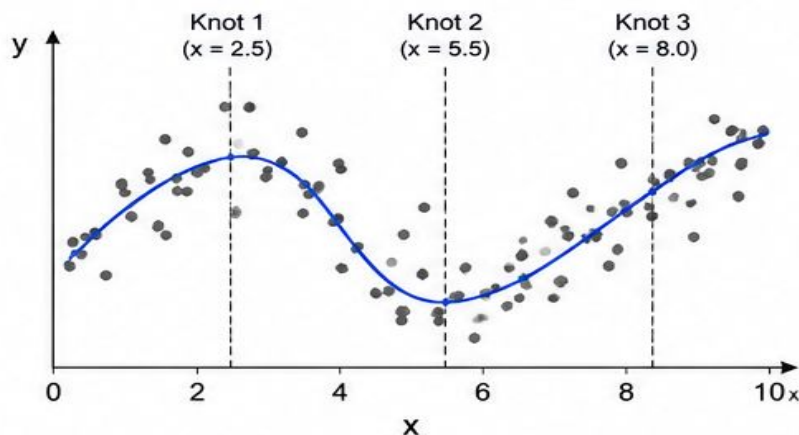
Key Points

- LOWESS is non-parametric (no global equation).
- Captures nonlinear trends effectively.
- Robust to outliers (less influenced by extreme values).
- Span (0–1) determines smoothness.
 - Common choice: span = 0.2 to 0.5

LOWESS = Locally Weighted Scatterplot Smoothing

1. REGRESSION SPLINES

Divides the input range into regions separated by knots and fits low-degree polynomials that join smoothly at boundaries.

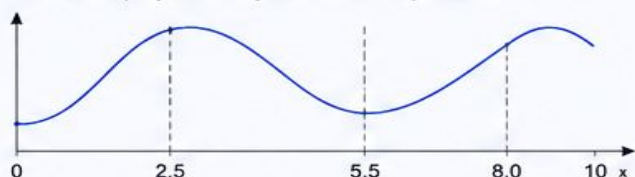


How it works

- Choose knot locations that split the input range.
- In each region between knots, fit a low-degree polynomial (e.g., degree 1 or 2).
- Polynomials are joined smoothly at the knots (continuous and have continuous first derivatives).

Example (piecewise cubic spline)

Piecewise polynomials joined smoothly at knots.

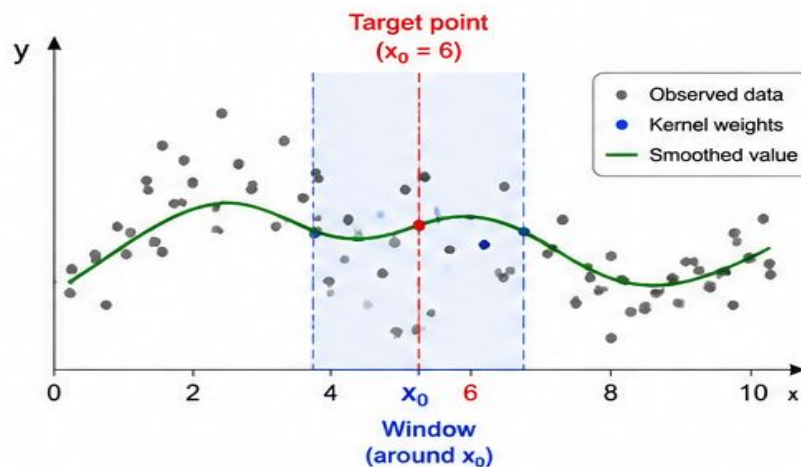


Key Points

- Flexible yet smooth fit.
- Knot placement controls flexibility.
- Commonly used in GAMs, interpolation, and trend modeling.

2. KERNEL SMOOTHING

Computes a weighted average of neighboring data points where closer points get higher weights based on a kernel function.



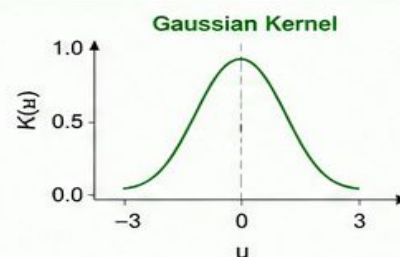
How it works

1. For target point x_0 , choose a bandwidth (window).
2. Assign weights to nearby points using a kernel function $K(\cdot)$.
Closer points $\rightarrow$ higher weights; farther $\rightarrow$ lower weights.
3. Compute weighted average to get the smoothed value.

Common kernel: Gaussian Kernel

$$K(u) = \frac{1}{\sqrt{2\pi}} \cdot e^{-u^2/2}$$

where $u = \frac{x - x_0}{h}$, h = bandwidth

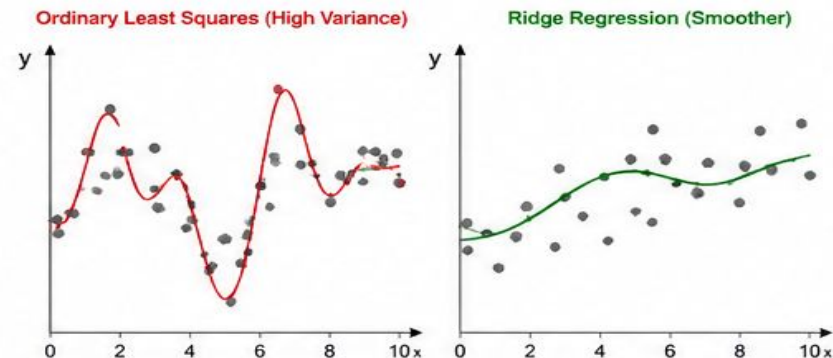


Key Points

- Non-parametric and very flexible.
- Bandwidth h controls smoothness (small $h \rightarrow$ wiggly; large $h \rightarrow$ smooth).
- Widely used in density estimation and regression smoothing.

3. REGULARIZED REGRESSION (RIDGE)

Smooths coefficient estimates by adding a penalty term to reduce variance and prevent overfitting.



How it works

- Ridge adds a penalty on the size of coefficients.
- Shrinks coefficients towards zero $\rightarrow$ reduces variance.
- Results in a smoother and more stable model.

Ridge Regression Objective

Minimize:

$$RSS(\beta) + \lambda \sum_{j=1}^p \beta_j^2$$

where:

- $RSS(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2$ (residual sum of squares)
- β_j = model coefficients
- $\lambda \geq 0$ = regularization parameter (larger $\lambda \rightarrow$ more smoothing)

Effect of λ (regularization strength)



Key Points

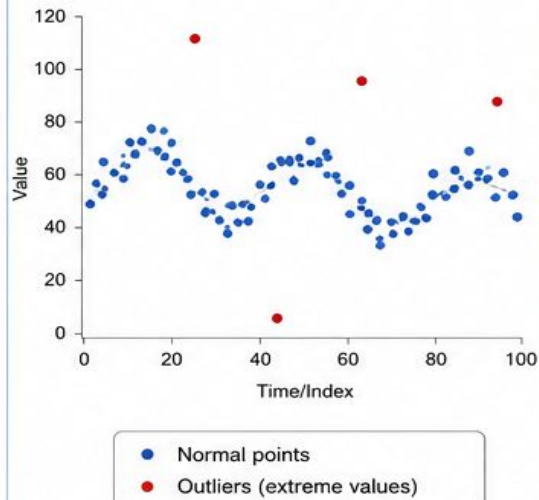
- Controls model complexity and prevents overfitting.
- Keeps all features but shrinks coefficients.
- Useful when predictors are many or highly correlated.

Technique	Main Idea	How it Smooths	Easy Keyword	Analogy
Kernel Smoothing	Looks at nearby data points	Takes a weighted average (closer points have more weight)	Neighbors	Ask nearby friends for directions. Trust the closest friend the most.
LOESS (LOWESS)	Looks at nearby data points	Fits a small regression line/curve to neighbors	Local Regression	Every neighborhood hires its own teacher to draw a small trend line.
Regression Splines	Divides data into sections	Fits a small polynomial in each section and joins them at knots	Knots (Sections)	Build a road using several curved pieces joined together.
Ridge Regression	Uses all the data	Adds a penalty to keep coefficients small and the model simple	Penalty	Put a speed limiter on a car so it doesn't go out of control.

Outlier Analysis for Smoothing – Easy Explanation

Outlier analysis means finding extreme (abnormal) data points and fixing them before applying a smoothing method.

1. Raw Data (with Outliers)



2. Outlier Detection

A. Z-Score Method

Compute Z-score: $Z = \frac{x - \mu}{\sigma}$
Flag as outlier if $|Z| > 3$

B. MAD Method (Robust)

MAD = median($|x_i - \text{median}(x)|$)
Flag as outlier if $\frac{|x_i - \text{median}(x)|}{\text{MAD}} > 3.5$

C. Clustering Method

Cluster similar values together.
Points not belonging to any cluster are treated as noise (outliers).

Red dot = Detected outliers

3. Fix or Handle Outliers

• Remove

Delete the outliers.

• Cap / Winsorize

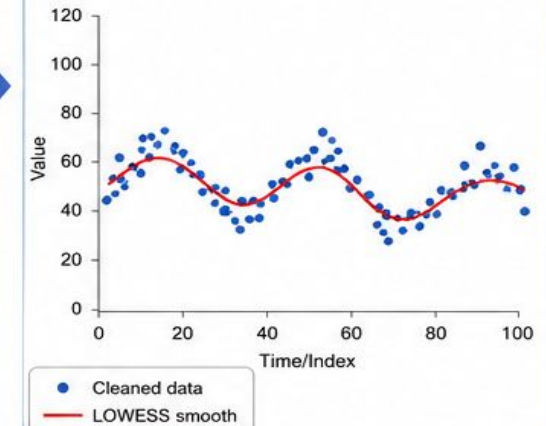
Replace with nearest acceptable value.

• Impute

Replace with median or predicted value.

4. Apply Smoothing (LOWESS)

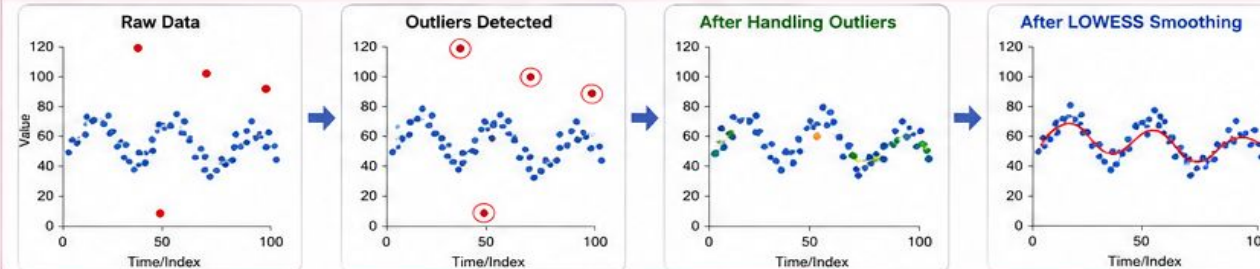
Use LOWESS to get a smooth trend after removing/handling outliers.



How Outlier Analysis Works (In Simple Steps)

- 1 Find extreme values using methods (Z-score, MAD, or Clustering).
- 2 Mark those points as outliers.
- 3 Fix them: remove, cap, or replace.
- 4 Now the data is clean.
- 5 Apply smoothing (like LOWESS) to get a clear trend.

Example Walk-through



Common Outlier Detection Methods – Quick Guide

Z-Score

Works well for normally distributed data.
Threshold: $|Z| > 3$

MAD (Median Absolute Deviation)

Robust to extreme values.
Threshold: > 3.5

Clustering

Group similar points.
Points outside clusters = outliers.

Key Takeaways

- ✓ Outliers can distort the trend.
- ✓ Detect them first.
- ✓ Handle them properly.
- ✓ Then apply smoothing (like LOWESS) for the best results.



One-Line Summary: Find extreme points → Handle them → Then smooth the data to see the real pattern.

Handling Outliers

Statistical methods: flag points beyond a threshold, e.g., more than $1.5 \times \text{IQR}$ from the quartiles, or $|\text{z-score}| > 3$ — simple and interpretable, but assumes a roughly known distribution.

Clustering-based detection: points that fall outside any cluster are treated as outliers — flexible for multi-dimensional data.

Visualization: box plots and scatter plots surface extreme points for human review.

Treatment: remove, cap (winsorize), transform (e.g., take the log), or keep and model separately — the choice depends on whether the outlier is an error or a real event worth preserving.

Encoding Categorical Variables

One-hot encoding: a binary column per category — no false order implied, but adds many columns at high cardinality.

Label (ordinal) encoding: maps categories to integers — compact, but implies an order that may not exist for nominal data.

Binary / hashing encoding: categories as binary codes or hashed buckets — more compact than one-hot at high cardinality.

Frequency encoding: replaces a category with an outcome statistic — powerful, but risks leaking outcome information.

Color		Red	Green	Blue
Red		1	0	0
Green		0	1	0
Blue		0	0	1
Green		0	1	0

Color		Color
Red		0
Green		1
Blue		2
Red		0

City (Original)	Integer Map	Binary Format
New York	1	001
Los Angeles	2	010
Chicago	3	011
Houston	4	100
Phoenix	5	101

Color		Color	Frequency
Red		Red	1
Green		Green	2
Blue		Blue	1
Green			

Feature Scaling

Min-Max Scaling

Rescales values into a fixed range, typically [0, 1]. Preserves the shape of the original distribution and relationships between values, but is sensitive to outliers, which can compress the rest of the data into a narrow band.

Standardization (Z-Score)

Centers values to mean 0 and standard deviation 1. Less thrown off by outliers than min-max, but does not bound values to a fixed range — useful when an algorithm assumes roughly normally distributed input.

Why it matters: distance-based methods (k-NN, k-means) and gradient-based methods (linear/logistic regression, neural networks) let large-range features dominate unless every feature is on a comparable scale.

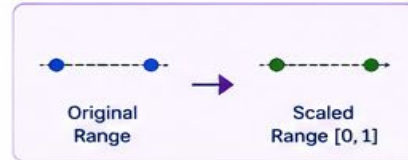
MIN-MAX SCALING / NORMALIZATION

Scale features to a fixed range [0, 1]

1. WHAT IS MIN-MAX SCALING?

Min-Max Scaling (also called **Normalization**) transforms numerical features to a fixed range, usually [0, 1].

It **preserves the relationship (order)** between values.



2. FORMULA

The Min-Max Scaling formula is:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \times (\text{max}_{\text{range}} - \text{min}_{\text{range}}) + \text{min}_{\text{range}}$$

Where:

- x = original value
- $x_{\min}$ = minimum value in the feature
- $x_{\max}$ = maximum value in the feature
- $\text{min}_{\text{range}}$ = lower limit of desired range (usually 0)
- $\text{max}_{\text{range}}$ = upper limit of desired range (usually 1)

For scaling to [0, 1]:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

(Here, $\text{min}_{\text{range}} = 0$, $\text{max}_{\text{range}} = 1$)

3. EXAMPLE

Suppose we have a dataset with 2 features: **Age (years)** and **Salary (in ₹)**

Original Data

Person	Age	Salary (₹)
A	25	25,000
B	30	40,000
C	35	60,000
D	40	80,000
E	50	100,000

Step 1: Find min and max

Age

Min ($x_{\min}$) = 25
Max ($x_{\max}$) = 50

Salary

Min ($x_{\min}$) = 25,000
Max ($x_{\max}$) = 100,000

Step 2: Apply Min-Max Scaling (to [0, 1])

$\text{Age}_{\text{scaled}} = \frac{\text{Age} - 25}{50 - 25} = \frac{\text{Age} - 25}{25}$

$\text{Salary}_{\text{scaled}} = \frac{\text{Salary} - 25,000}{100,000 - 25,000} = \frac{\text{Salary} - 25,000}{75,000}$

Step 3: Scaled Data

Person	Age (Original)	Age (Scaled) [0,1]	Salary (Original ₹)	Salary (Scaled) [0,1]
A	25	0.00	25,000	0.00
B	30	0.20	40,000	0.20
C	35	0.40	60,000	0.47
D	40	0.60	80,000	0.73
E	50	1.00	100,000	1.00



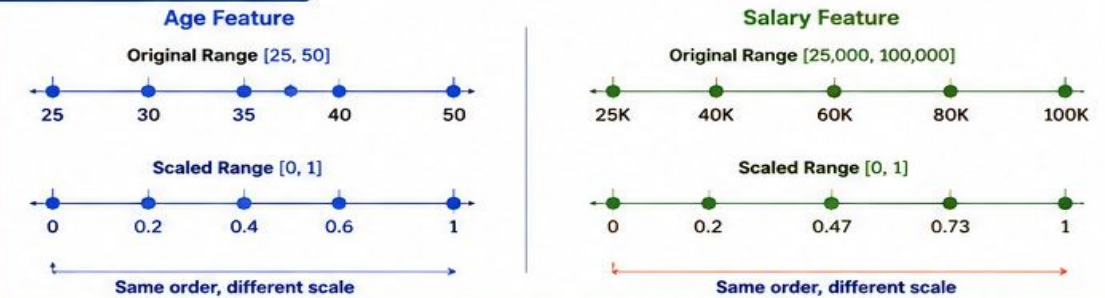
Key Point:

All values are now in the range [0, 1].
The relative order of values is preserved.

4. STEP-BY-STEP CALCULATION (DETAILED EXAMPLE)

Person	Age (x)	Calculation	Age scaled	Salary (x)	Calculation	Salary scaled
A	25	$\frac{(25 - 25)}{(50 - 25)} = 0$	0.00	25,000	$\frac{(25,000 - 25,000)}{(100,000 - 25,000)} = 0$	0.00
B	30	$(30 - 25) / 25 = 0.20$	0.20	40,000	$(40,000 - 25,000) / 75,000 = \frac{15,000}{75,000}$	0.20
C	35	$(35 - 25) = 10 / 25 = 0.40$	0.40	60,000	$(60,000 - 25,000) / 75,000 = \frac{35,000}{75,000}$	0.47
D	40	$(40 - 25) = 15 / 25 = 0.60$	0.60	80,000	$(80,000 - 25,000) / 75,000 = \frac{55,000}{75,000}$	0.73
E	50	$(50 - 25) = 25 / 25 = 1.00$	1.00	100,000	$(100,000 - 25,000) / 75,000 = \frac{75,000}{75,000}$	1.00

5. VISUAL INTUITION



6. WHY USE MIN-MAX SCALING?

- ✓ Brings all features to a common scale.
- ✓ Useful for algorithms that depend on distance (e.g., KNN, SVM, K-Means).
- ✓ Helps in faster convergence of gradient-based models.
- ✓ Preserves the original distribution shape (linear transformation).

7. ADVANTAGES

- ✓ Simple and easy to implement.
- ✓ Keeps all values in a fixed range and small range.
- ✓ Preserves relationships between values.
- ✓ Works well when there are no extreme outliers.

8. LIMITATIONS

- ⚠ Sensitive to outliers.
- ⚠ If new data has values outside the range, scaled values can be < 0 or > 1 .
- ⚠ Distribution shape remains the same.

9. WHEN TO USE?



Use Min-Max Scaling when:

- You know the data does not have extreme outliers.
- You need values in a fixed range (e.g., 0 to 1).
- The algorithm requires features on a similar scale.

10. KEY TAKEAWAY



Min-Max Scaling transforms data to a **fixed range [0, 1]** without changing the relative relationship between values.



In Short:



Find Min & Max
of the feature



Apply Formula



Get Scaled Values
in [0, 1]



Better Performance &
Faster Learning

STANDARDIZATION (Z-SCORE SCALING)

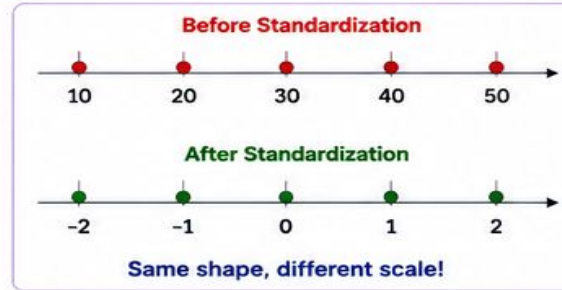
Transform features to have mean 0 and standard deviation 1

1. WHAT IS STANDARDIZATION?

Standardization (or Z-score Scaling) transforms numerical features so that they have:

- ✓ Mean = 0
- ✓ Standard Deviation = 1

It keeps the data centered around 0 and scales it based on its spread (standard deviation).



2. FORMULA

The Standardization formula (Z-score formula):

$$z = \frac{x - \mu}{\sigma}$$

What does it do?

It shows how many standard deviations a value is away from the mean.

- $z = 0$ → value is equal to the mean
- $z > 0$ → value is above the mean
- $z < 0$ → value is below the mean

Where:

- x = original value
- μ = mean (average) of the feature
- σ = standard deviation of the feature
- z = standardized (scaled) value

Properties after Standardization

- ✓ Mean of all z-values = 0
- ✓ Standard deviation of z-values = 1
- ✓ No change in the shape or order of data

3. EXAMPLE (STEP-BY-STEP)

Example: Heights (in cm) of 5 students

Original Data	
Student	Height (x)
A	150
B	160
C	170
D	180
E	190

Step 1: Calculate Mean (μ)

$$\mu = \frac{150 + 160 + 170 + 180 + 190}{5} = \frac{850}{5} = 170$$

Step 2: Calculate Standard Deviation (σ)

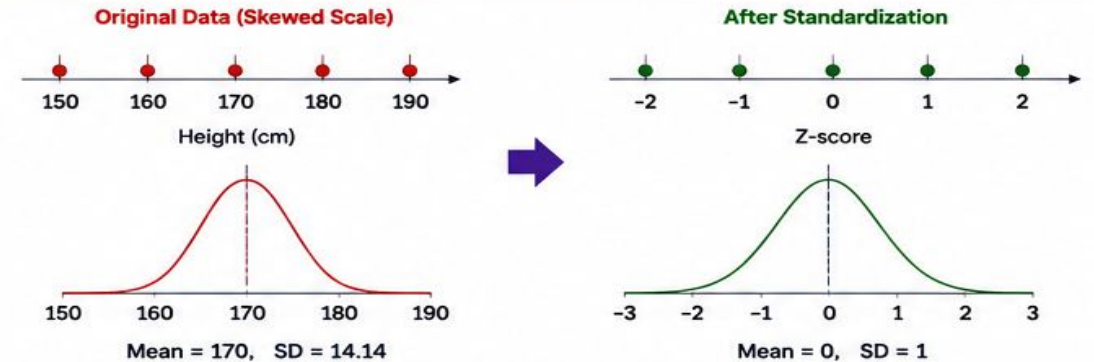
$$\sigma = \sqrt{\frac{\sum (x - \mu)^2}{n}} = \sqrt{\frac{1000}{5}} = \sqrt{200} = 14.14$$

Step 3: Apply Standardization				
Student	x (Height)	$x - \mu$	$z = \frac{x - \mu}{\sigma}$	z (Rounded)
A	150	-20	-20/14.14	-1.41
B	160	-10	-10/14.14	-0.71
C	170	0	0/14.14	0.00
D	180	10	10/14.14	0.71
E	190	20	20/14.14	1.41

💡 Observe:

- ✓ Mean of z-values = 0.00
- ✓ Standard deviation of z-values ≈ 1
- ✓ Order is same as original data

4. VISUAL UNDERSTANDING



Interpretation: A z-score of 1.41 means the value is 1.41 standard deviations above the mean.

5. WHEN TO USE STANDARDIZATION?



Algorithms that use distance or gradient: KNN, K-Means, SVM, PCA, Gradient Descent, Neural Networks



When features have different units or scales (e.g., Height in cm vs Weight in kg)



When data is roughly normally distributed

6. ADVANTAGES

- ✓ Centers data around 0.
- ✓ Helps in faster convergence of machine learning models.
- ✓ Works well even if data has negative values.
- ✓ Less sensitive to outliers than Min-Max Scaling.

7. LIMITATIONS

- ⚠ Not bound to a fixed range (values can be < 0 or > 1).
- ⚠ Sensitive to outliers because mean and std are affected.

8. STANDARDIZATION vs MIN-MAX SCALING

Aspect	Standardization (Z-score)	Min-Max Scaling
Resulting Range	$(-\infty, \infty)$	[0, 1]
Mean	0	Not fixed
Std Deviation	1	Not fixed
Sensitive to Outliers	Yes (affected)	Yes (compressed)
Best For	Most ML algorithms	When fixed range is needed (e.g., in Neural Networks)

9. KEY TAKEAWAY



Standardization converts data using its mean and standard deviation so that the new data has a mean of 0 and a standard deviation of 1, while preserving the original distribution shape and order.

10. QUICK SUMMARY (THE FLOW)



Step 1
Collect Data



Step 2
Calculate Mean (μ) and Std Dev (σ)



$$z = \frac{x - \mu}{\sigma}$$

Step 3
Apply Formula to each value



Step 4
Get Standardized Data (mean = 0, std = 1)



Step 5
Use in ML Model



REMEMBER

Standardization is all about **CENTERING** (mean = 0) and **SCALING** (standard deviation = 1).

Discretization

Definition: converts a continuous attribute into a finite set of intervals, each treated as a category or ordinal label.

Binning: equal-width bins (fixed interval size) or equal-frequency bins (equal count per bin).

Histogram analysis: partitions values based on the distribution's peaks and valleys.

Cluster analysis: groups nearby values into intervals via a clustering algorithm.

Entropy-based (supervised): picks split points that best separate class labels.

Example: exact age into bins like "0–18," "19–35," "36–60," "60+" for a classification model.

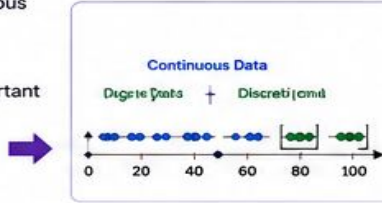
DISCRETIZATION

Converting Continuous Data into Discrete Intervals (Bins)

1. WHAT IS DISCRETIZATION?

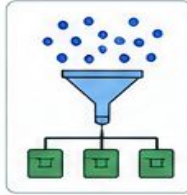
Discretization means converting a continuous numerical variable into a finite number of intervals or categories (bins).

It simplifies the data while preserving important information.



2. WHY DISCRETIZE?

- ✓ Simplifies data and reduces complexity.
- ✓ Improves the performance of some machine learning algorithms.
- ✓ Makes data more interpretable.
- ✓ Reduces the effect of noise and outliers.
- ✓ Needed in algorithms that require categorical input (e.g., Naive Bayes, Decision Trees).



3. COMMON DISCRETIZATION METHODS

 1. Equal Width (Interval Width)	Divide the range into intervals of equal size.	Best when data is uniformly distributed.
 2. Equal Frequency (Quantile)	Each interval (bin) contains (approximately) the same number of data points.	Best when data is skewed or not uniformly distributed.
 3. K-Means Discretization	Uses K-Means clustering to create k clusters and uses the cluster ranges as bins.	Best when data has natural clusters.
 4. Entropy-Based (Decision Tree)	Uses information gain (entropy) to find optimal cut points.	Best for predictive modeling (used in decision trees).
 5. Custom / Domain Knowledge	Bins are created based on expert knowledge or business rules.	Best when domain expertise is available.

4. EXAMPLE (STEP-BY-STEP)

Let's discretize the following continuous data (Age of customers):

Original Data (Age in years)

18	22	25	28	35	37	45	52	52	63	70	72	80	90
----	----	----	----	----	----	----	----	----	----	----	----	----	----

1) Equal Width (bin width = 20)

- Range = $90 - 18 = 72 \Rightarrow \text{Bin width} \approx 20$

Bins (Age)	18 – <38	38 – <58	58 – <78	78 – ≤ 90
Meaning	Young	Adult	Middle Aged	Senior
Discretized Data	18, 22, 25, 28, 35, 37	45, 52, 55	63, 70, 72	80, 85, 90

2) Equal Frequency (Quantile) (15 values $\Rightarrow$ 4 bins $\Rightarrow$ ~4 values per bin)

Bins (by count)	1 st Quartile	2 nd Quartile	3 rd Quartile	4 th Quartile
Range (Age)	18 – 25	28 – 37	45 – 63	70 – 90
Discretized Data	18, 22, 25	28, 35, 37	45, 52, 55, 63	70, 72, 80, 85, 90

3) K-Means Discretization (k = 3 clusters)

- After applying K-Means on the data, suppose we get 3 clusters:

Bins (Clusters)	Cluster 1	Cluster 2	Cluster 3
Range (Age)	18 – 28	35 – 55	63 – 90
Meaning	Young	Adult	Senior
Discretized Data	18, 22, 25, 28	35, 37, 45, 52, 55	63, 70, 72, 80, 85, 90

4) Entropy-Based (Decision Tree Approach)

- Uses information gain to find best split points.
- Suppose the best splits found are at 30, 50, 70.

Bins (Age)	≤ 30	30 – 50	50 – 70	> 70
Meaning	Very Young	Young Adult	Adult	Senior
Discretized Data	18, 22, 25, 28	35, 37, 45	52, 55, 63, 70	72, 80, 85, 90

5. BEFORE vs AFTER DISCRETIZATION



6. METHOD COMPARISON

Method	How it Works	Best When
Equal Width	Equal size intervals	Data is fairly uniform
Equal Frequency (Quantile)	Equal number of values in each bin	Data is skewed
K-Means	Cluster data and use cluster ranges	Data has natural clusters
Entropy-Based	Finds optimal split points using entropy	For predictive modeling (trees)
Custom / Domain Knowledge	Bins based on expert knowledge	Domain knowledge is available

7. WHEN TO USE DISCRETIZATION?

- ★ When algorithms require categorical input.
- ★ When you want to reduce the number of unique values.
- ★ When data contains noise or small variations.
- ★ When improving model interpretability.
- ★ In credit scoring, risk analysis, medical diagnosis, customer segmentation, weather analysis, etc.

8. ADVANTAGES

- ✓ Simplifies complex data.
- ✓ Improves model performance for some algorithms.
- ✓ Reduces noise and outliers' impact.
- ✓ Makes results easier to understand and explain.

9. LIMITATIONS

- ⚠ May lead to loss of information.
- ⚠ Choice of bins can affect results.
- ⚠ Too many bins $\rightarrow$ overfitting.
- ⚠ Too few bins $\rightarrow$ underfitting.

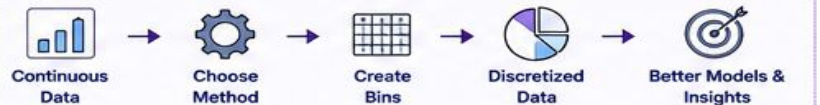
10. KEY TAKEAWAY



Discretization converts continuous values into meaningful intervals (bins). It makes data simpler, more interpretable, and suitable for many machine learning algorithms.

Right choice of method and number of bins is important!

QUICK SUMMARY



Summary

TASK	ADDRESSES	COMMON TECHNIQUES
Missing data	Absent or unrecorded values	Deletion, mean/median fill, prediction
Noise & outliers	Measurement error and extreme values	Binning, regression, IQR/z-score detection
Categorical encoding	Non-numeric attributes	One-hot, ordinal, target encoding
Scaling, normalization & discretization	Differing scales and continuous ranges	Min-max, z-score, decimal scaling, binning

Introduction to EDA

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is the process of analyzing datasets to summarize their main characteristics using visual and statistical methods. Introduced by John Tukey in the 1970s, EDA emphasizes the importance of visual exploration and understanding of data before applying machine learning models.

Primary Goals

To understand the underlying structure of the data.

To identify outliers and missing values.

To determine relationships between variables.

To generate hypotheses for further analysis.

EDA Importance in Machine Learning

EDA serves as the foundation for building robust and accurate machine learning models. Here are the key reasons why EDA is indispensable:



**DATA QUALITY
ASSESSMENT**

EDA helps identify issues such as missing values, duplicate records, and inconsistencies in data.

**FEATURE SELECTION
AND ENGINEERING**

By analyzing correlations and distributions, EDA aids in selecting relevant features and engineering new ones.

**MODEL
SELECTION**

Insights from EDA can influence the choice of machine learning algorithms.

**DATA
VISUALIZATION**

Visualization during EDA communicates complex relationships and patterns effectively.

**ANOMALY
DETECTION**

It highlights unusual data points that could skew model performance.

Exploratory Data Analysis (EDA)

- **Definition:** Exploratory Data Analysis (EDA) is the process of analyzing datasets to summarize main characteristics before modeling.
- **Purpose:**
 - Identifies patterns and trends
 - Detects outliers and missing values
 - Assists in feature engineering and selection
- Exploratory Data Analysis (EDA) is a fundamental step in ML before modeling. The main objectives include:
- **Understanding data characteristics** (patterns, distributions, missing values).
- **Detecting anomalies, trends, and relationships** to improve model performance.
- **Preparing data** by handling missing values, feature engineering, and scaling.

EDA ensures that we make informed decisions before applying ML algorithms.

Key Steps in Exploratory Data Analysis

- Inspecting Data Types
- Checking Data Dimensions
- Viewing Sample

Understanding the Dataset

Summarizing the Data

- Mean, Median, Mode
- Standard Deviation and Variance
- Minimum, Maximum, and Range
- Skewness and Kurtosis

- Handling Missing Values
- Removing Duplicates
- Addressing Outliers

Data Cleaning

Visualizing Data

- Histograms
- Box Plots
- Scatter Plots
- Heatmaps
- Bar Charts and Pie Charts

- Correlation Analysis
- Cross-tabulation
- Pair Plots

Exploring Relationships Between Variables

Visualizing Relationships

- Scaling and Normalization
- Log Transformation
- One-Hot Encoding
- Feature Interaction

Classification Exploratory Data Analysis

- **EDA**

- **Univariate :**

- Graphical EDA: Histograms, Boxplots
 - Non-graphical EDA: Frequency distribution table, measures of central tendency, measures of dispersion

- **Bivariate**

- Graphical EDA
 - Non Graphical EDA

- **Multivariate**

- Graphical EDA: Scatter plot, Heat Map
 - Non-graphical EDA: Cross-tabulation, correlation matrix

Example: EDA in Action

Let's walk through a basic EDA process using Python:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
data = pd.read_csv("data.csv")

# Summarize data
print(data.info())
print(data.describe())

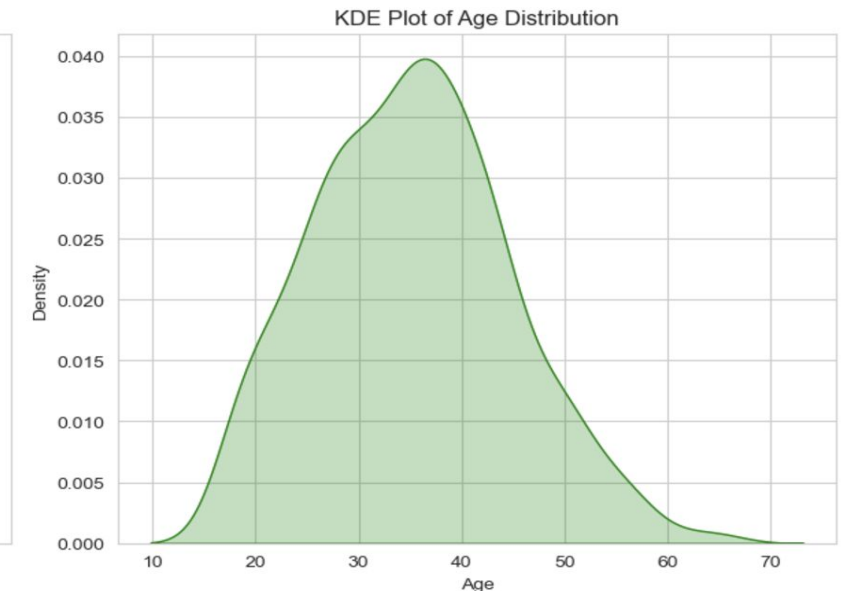
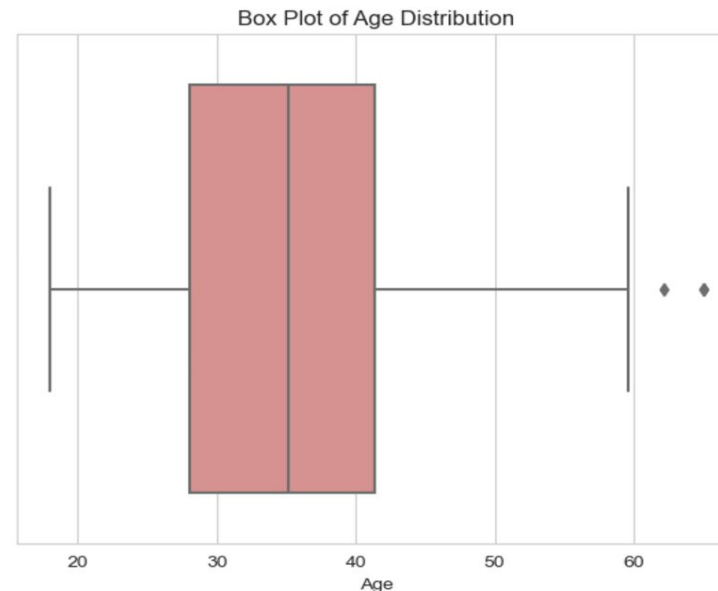
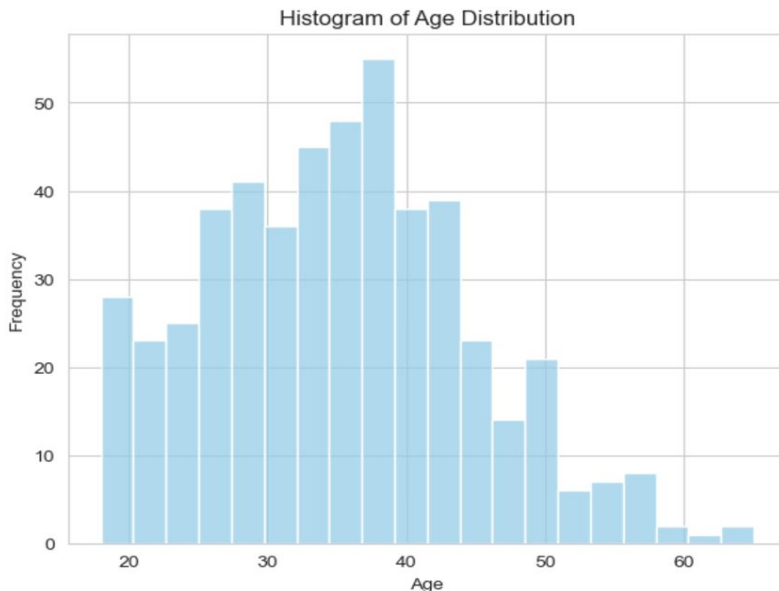
# Check for missing values
print(data.isnull().sum())
```

EDA can be divided into different types based on the scope of analysis:

1. **Univariate Analysis:** Examines single variables.
2. **Bivariate Analysis:** Analyzes relationships between two variables.
3. **Multivariate Analysis:** Examines multiple variables together.
4. **Missing Value Analysis:** Identifies and handles missing data.
5. **Outlier Detection:** Detects extreme values.
6. **Feature Engineering Analysis:** Transforms data to improve modeling.
7. **Time-Series Analysis:** Identifies trends, seasonality, and patterns in temporal data.
8. **Data Imbalance Analysis:** Checks class distribution in classification problems.

Univariate Analysis

- Univariate analysis explores the **distribution of a single variable** in a dataset.
- **Techniques:**
 - **Summary Statistics:** Mean, Median, Mode, Standard Deviation,
 - **Visualizations:**
 - **Histogram:** Shows frequency distribution.
 - **Box Plot:** Detects outliers and spread.
 - **KDE(Kernel density estimation) Plot:** Estimates probability density.
- Example: Analyzing age distribution in a dataset using histograms.



Univariate Non-graphical:

This is the simplest form of data analysis as during this we use just one variable to research the info. The standard goal of univariate non-graphical EDA is to know the underlying sample distribution/ data and make observations about the population. Outlier detection is additionally part of the analysis. The characteristics of population distribution include:

- **Central tendency:** The central tendency or location of distribution has got to do with typical or middle values. The commonly useful measures of central tendency are statistics called mean, median, and sometimes mode during which the foremost common is mean. For skewed distribution or when there's concern about outliers, the median may be preferred.
- **Spread:** Spread is an indicator of what proportion distant from the middle we are to seek out the find the info values. the quality deviation and variance are two useful [measures of spread](#). The variance is that the mean of the square of the individual deviations and therefore the variance is the root of the variance
- **Skewness and kurtosis:** Two more useful univariates descriptors are the skewness and kurtosis of the distribution. Skewness is that the measure of asymmetry and kurtosis may be a more subtle measure of peakedness compared to a normal distribution

Frequency distribution

- It is a way of organizing your data so that it makes more sense.
- It can be shown in a table or a graph.
- Frequency distribution of ungrouped data
 - Below are marks of 20 students in maths out of 2
 - [21,23,19,17,12,15,15,17,17,19,23,23,21,23,25,21,19,19,19]

Marks Obtained	Tally Marks	Frequency
12	I	1
15	II	2
17	III	3
19	IIII	5
21	III	3
23	IIII	4
25	II	2

- Frequency distribution of grouped data
 - The presentation of the earlier data can be expressed into groups. These groups are called classes or the class interval.
 - Each class interval is bounded by two figures called the class limits.

Marks	Number of Students (Frequency)
0 - 10	0
10 - 20	11
20 - 30	9

A **frequency table** is a simple way to display the number of occurrences of a particular value or characteristic.

- **The absolute frequency** describes the number of times a particular value for a variable (data item) has been observed to occur.
- **The relative frequency** describes the number of times a particular value for a variable (data item) has been observed to occur in relation to the total number of values for that variable.
[Class frequency / Total frequency]

Height of Children

Height (cm) of children	Absolute frequency	Relative frequency
120 – less than 130	9	18%
130 – less than 140	10	20%
140 – less than 150	13	26%
150 – less than 160	11	22%
160 – less than 170	7	14%
Total	50	100%

Central Tendency

- **Measures of Central Tendency** gives us an idea about the concentration of the values in the central part of the distribution of frequencies of the whole data.
- The term central tendency refers to the “middle” value or perhaps a typical value of the data, and is measured using the mean, median, or mode.
- Each of these measures is calculated differently, and the one that is best to use depends upon the situation.

Mean(Arithmetic Mean)

- The mean is the most commonly used measure of central tendency. When we talk about an “average,” we usually refer to the mean.
- The mean is simply the sum of the values divided by the total number of items in the set. The result is referred to as the arithmetic mean.
- Sometimes it is useful to give more weightage to certain data points. In that case, the result is called the weighted arithmetic mean.
- Example: , 11, 7

Step 1: Add the numbers $6 + 11 + 7 = 24$

Step 2: Divide by how many numbers there are : $24 \div 3 = 8$

The Mean is 8.

Median

The Median is the “middle” of a sorted list of numbers.

It is the middlemost value of the data arranged in ascending or descending order of magnitude.

Thus, median is a positional average

Example 1: Given numbers: 3, 13, 7, 5, 21, 23, 39, 23, 40, 23, 14, 12, 56, 23, 29

Step 1: Arrange the numbers in ascending order

3, 5, 7, 12, 13, 14, 21, 23, 23, 23, 23, 29, 39, 40, 56

Step 2: Find the middle number

There are 15 numbers. Since 15 is odd, the median is the 8th number.

3, 5, 7, 12, 13, 14, 21, 23, 23, 23, 23, 29, 39, 40, 56

Therefore, the median value = 23.

Example 2: If Two Numbers in the Middle

3, 13, 7, 5, 21, 23, 23, 40, 23, 14, 12, 56, 23, 29

Step 1: Arrange the numbers in ascending order: 3, 5, 7, 12, 13, 14, 21, 23, 23, 23, 23, 29, 40, 56

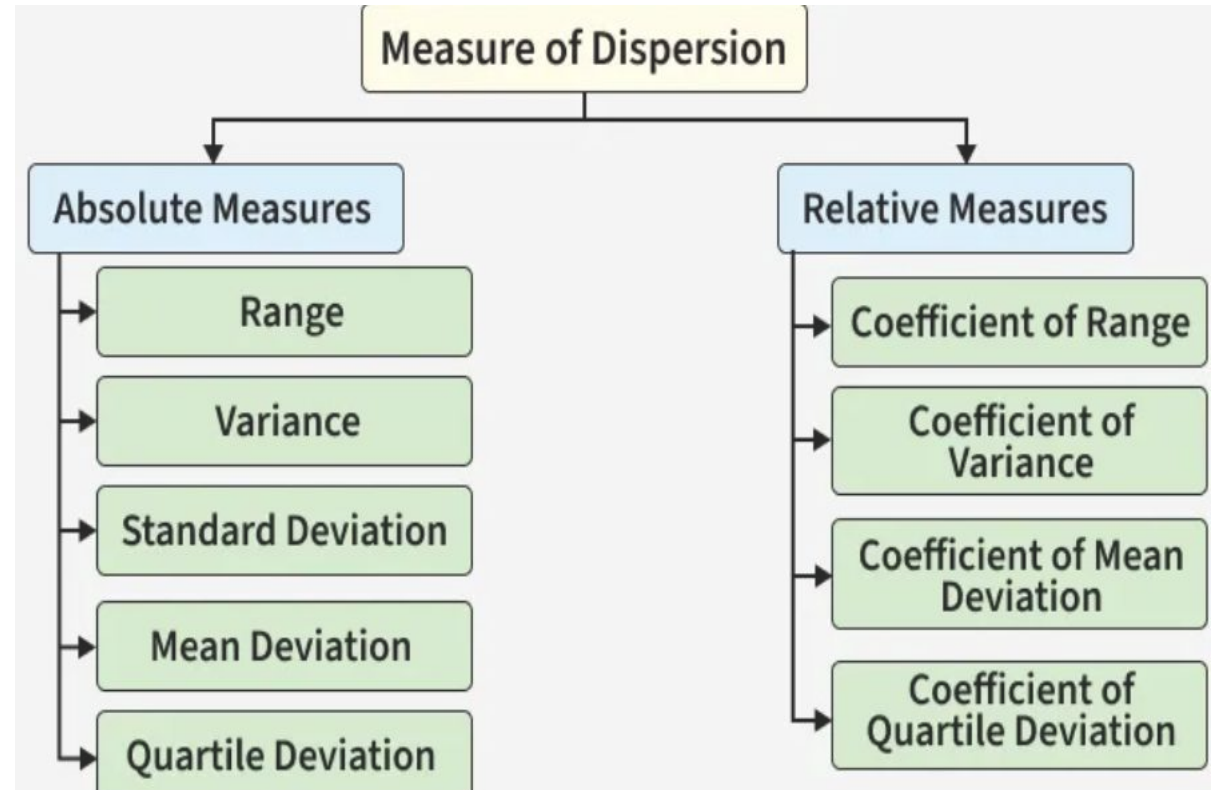
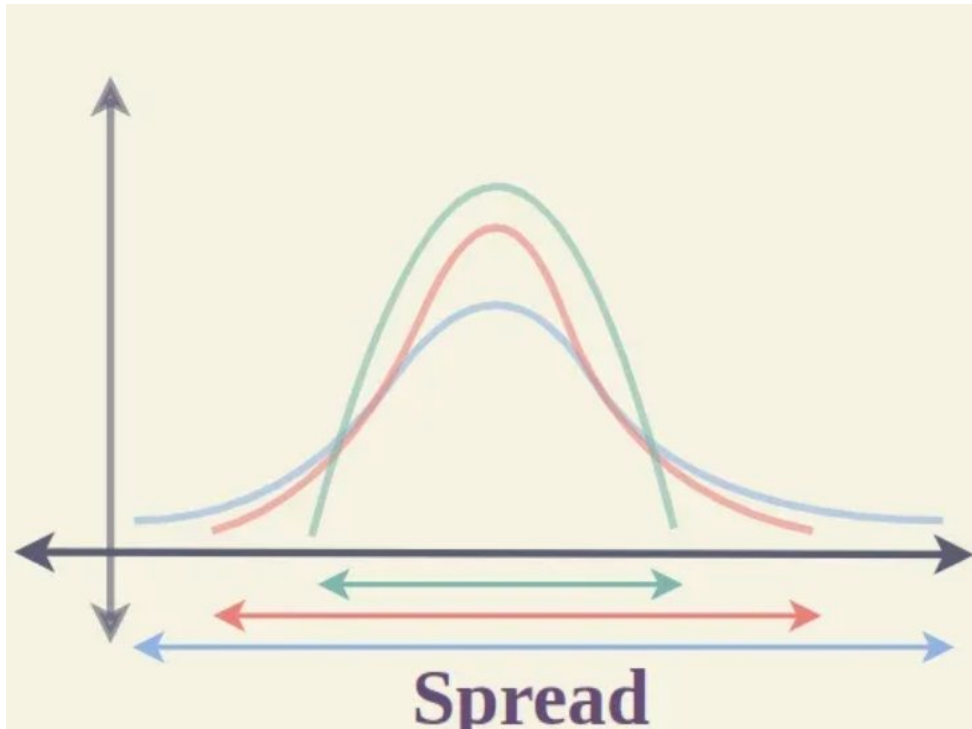
Step 2: Find the two middle numbers: there are two middle numbers: 21 and 23

Step 3: Find the average of the two middle numbers $\text{Median} = (21 + 23) / 2 = 22$

Therefore, the Median = 22.

Measures of dispersion

- Dispersion refers to the degree to which data values in a dataset are spread out or scattered around an average (such as the mean or median). It helps us understand the variability or consistency within the data — whether the values are closely grouped or widely spread.
- In statistics, measures of dispersion are numerical values that describe the extent of variation among data points. They provide insight into how the data is distributed and help compare the reliability or uniformity of different datasets.



Absolute Measure of Dispersion: The measures of dispersion that are measured and expressed in the units of data themselves are called the Absolute Measure of Dispersion. For example - Meters, Dollars, Kg, etc.

- **Range:** It is defined as the difference between the largest and the smallest value in the distribution. The measure of dispersion is used to variability of the data.
- **Mean deviation:** It is the arithmetic mean of the differences between the values and their mean.
- **Standard Deviation:** It is the square root of the arithmetic average of the squares of the deviations measured from the mean.
- **Variance:** It is defined as the average of the square deviation from the mean of the given data set.
- **Quartile Deviation:** It is defined as half of the difference between the third quartile and the first quartile in a given data set.
- **Interquartile Range:** The difference between the upper(Q_3) and lower(Q_1) quartiles is called the Interquartile Range. Its formula is given as $Q_3 - Q_1$.

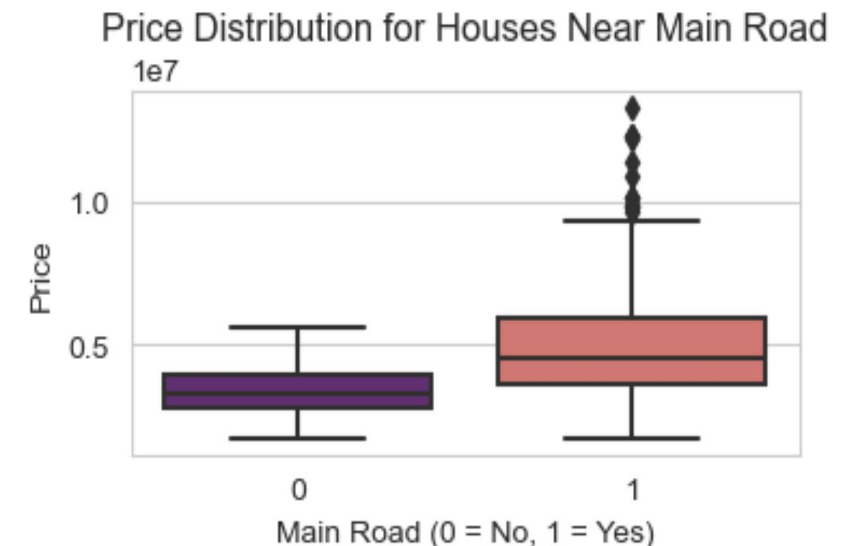
Relative Measure of Dispersion: We use relative measures of dispersion to measure the two quantities that have different units to get a better idea of the scattering of the data.

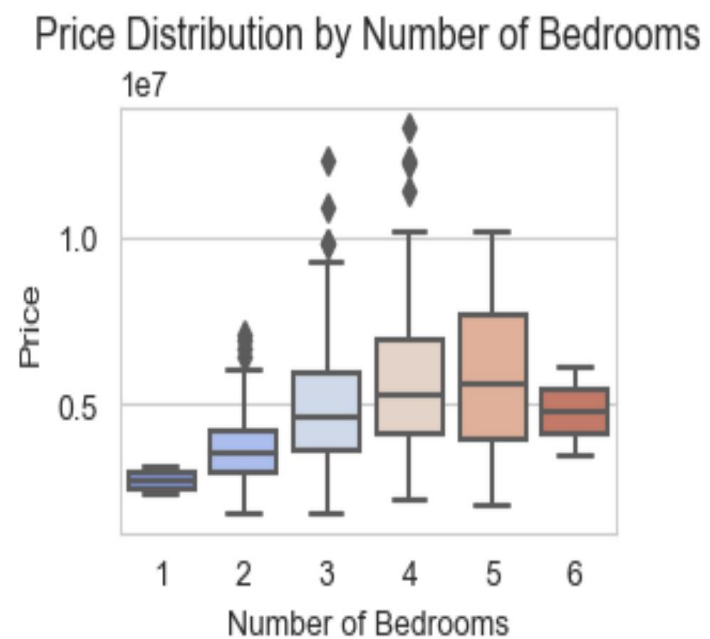
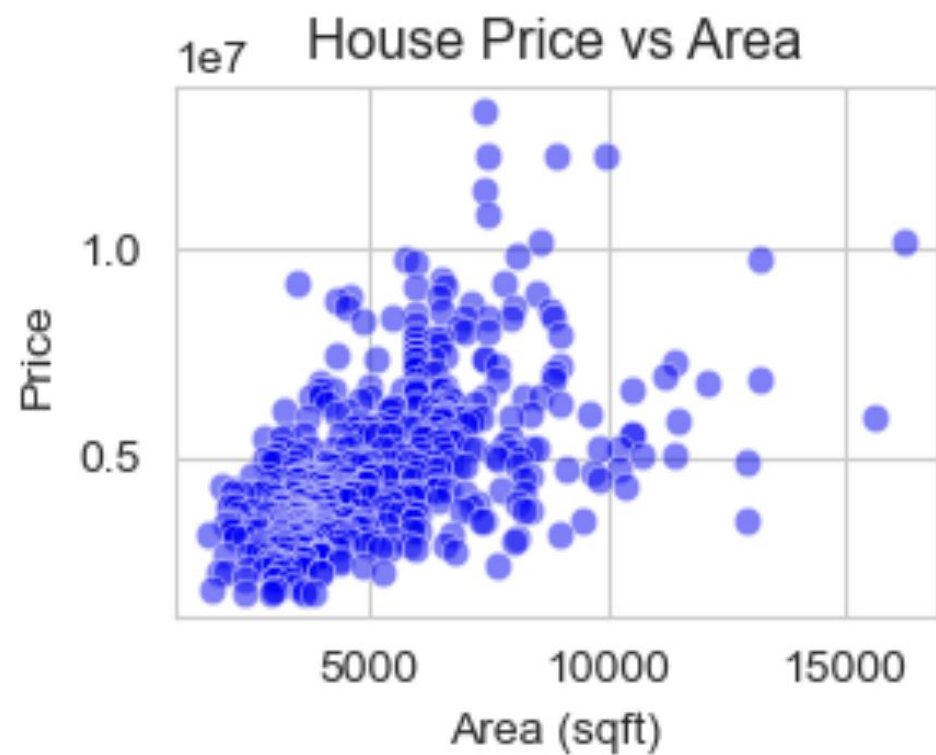
- **Coefficient of Range:** It is defined as the ratio of the difference between the highest and lowest value in a data set to the sum of the highest and lowest value.
- **Coefficient of Variation:** It is defined as the ratio of the standard deviation to the mean of the data set. We use percentages to express the coefficient of variation.
- **Coefficient of Mean Deviation:** It is defined as the ratio of the mean deviation to the value of the central point of the data set.

Measures of Dispersion	Formulas
Range	H - S where H is the Largest Value and S is the Smallest Value
Variance	Population Variance, $\sigma^2 = \Sigma(x_i - \mu)^2 / n$ Sample Variance, $S^2 = \Sigma(x_i - \bar{x})^2 / (n-1)$ where μ is the mean and n is the number of observations
Standard Deviation	S.D. = $\sqrt{(\sigma^2)}$
Mean Deviation	$M.D. = \Sigma x_i - \bar{x} / n$ where, a is the central value(mean, median, mode)and n is the number of observation
Quartile Deviation	$(Q_3 - Q_1)/2$ where, Q_3 = Third Quartile and Q_1 = First Quartile
Coefficient of Range	$(H - S)/(H + S)$
Coefficient of Variation	$C.V. = S/\bar{x} \times 100$ or $C.V. = \sigma/\mu \times 100$
Coefficient of Mean Deviation	$(\text{Mean Deviation})/\bar{x}$ where, μ is the central point for which the mean is calculated
Coefficient of Quartile Deviation	$(Q_3 - Q_1)/(Q_3 + Q_1)$

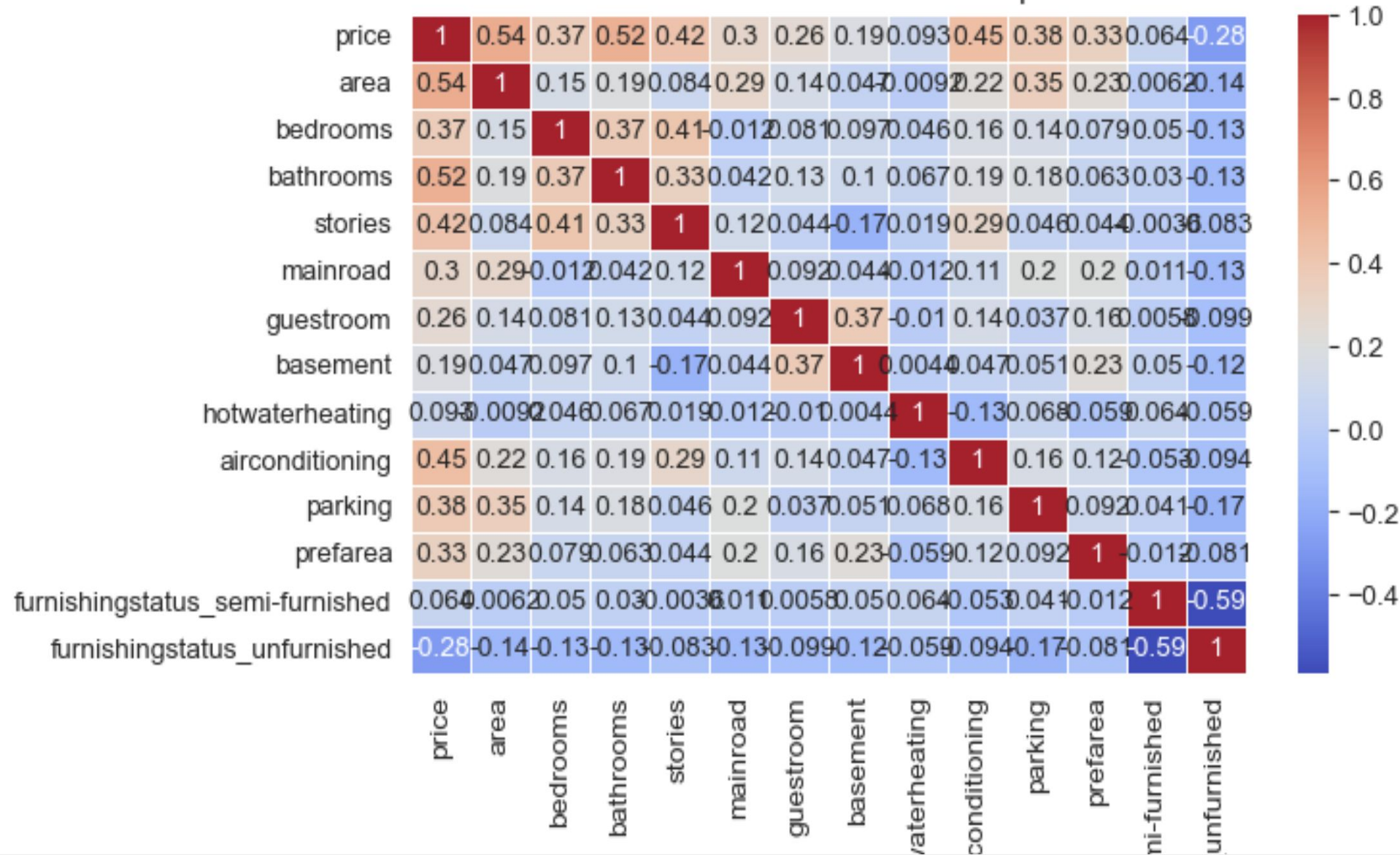
Bivariate Analysis

- Bivariate analysis examines the relationship between two variables using different approaches:
 - **Numerical-Numerical:**
 - **Scatter Plot:** Visualizes correlations.
 - **Correlation Matrix:** Measures relationships using Pearson/Spearman coefficients.
 - **Categorical-Categorical:**
 - **Bar Charts:** Shows categorical distribution.
 - **Numerical-Categorical:**
 - **Box Plots:** Compares distributions across categories.
- Example: Checking how house price varies with square feet using a scatter plot.





Feature Correlation Heatmap



Multivariate Analysis

- This analysis evaluates **the interactions between multiple variables** simultaneously.
- **Graphical Multivariate EDA:**
 - **Pair Plot:** Visualizes pairwise relationships.
 - **Heatmap:** Displays correlations using a color matrix.
 - **PCA (Principal Component Analysis):** Reduces dimensions for better visualization.
- Example: Finding correlations between age, salary, and spending habits in customer datasets.
- **Non-graphical Multivariate EDA:** Multivariate non-graphical EDA technique is usually used to show the connection between two or more variables within the sort of either cross-tabulation or statistics.
- For categorical data, an extension of tabulation called cross-tabulation is extremely useful. For 2 variables, cross-tabulation is preferred by making a two-way table with column headings that match the amount of one-variable and row headings that match the amount of the opposite two variables, then filling the counts with all subjects that share an equivalent pair of levels.
- For each categorical variable and one quantitative variable, we create statistics for quantitative variables separately for every level of the specific variable then compare the statistics across the amount of categorical variable.

Missing Value Analysis

- Missing values can distort model predictions, so detecting and handling them is crucial.
- **How to detect missing values?**
 - Use `.isnull().sum()` in Pandas to count missing entries.
 - **Seaborn Heatmap**: Highlights missing data patterns.
- **Handling missing values:**
 - **Drop missing values** if a small percentage is affected.
 - **Impute values:**
 - **Mean/Median/Mode imputation** for numerical data.
 - **Predictive imputation** using regression models.
- Example: Handling missing **income** values in a customer dataset by filling them with the **median income**.

Outlier Detection

Identifies extreme values affecting model performance

- Outliers can skew model results and need careful handling.
- **Common detection techniques:**
 - **Box Plots:** Visualize extreme values.
 - **Scatter Plots:** Reveal unusual data points.
- **Handling outliers:**
 - Remove extreme values (if due to errors).
 - Use robust models (like Decision Trees).
- Example: Identifying outliers in salary data that could be errors or genuine high earners

Feature Engineering Analysis

- Feature engineering enhances model performance by transforming raw data.
- **Techniques:**
 - **Feature Encoding Categorical Variables:**
 - One-Hot Encoding (for categorical data).
 - Label Encoding (for ordinal data).
 - **Feature Scaling:**
 - Standardization (Z-score scaling).
 - Normalization (Min-Max scaling).
- Example: Transforming house age into categorical bins like Old, Medium, New.

Feature Selection Techniques

Feature selection is a critical aspect of machine learning that involves choosing the most relevant features from a dataset. It plays a significant role in improving model performance, reducing overfitting, and enhancing interpretability. In this article, I will explore different feature selection techniques, their importance, utility, applicability, and provide code examples to illustrate each technique.

The goal of feature selection is to improve the performance of a machine learning model by reducing the dimensionality of the input data and removing irrelevant or redundant features.

All Features



Informative Features Selection



Final Features



Importance of Feature Selection

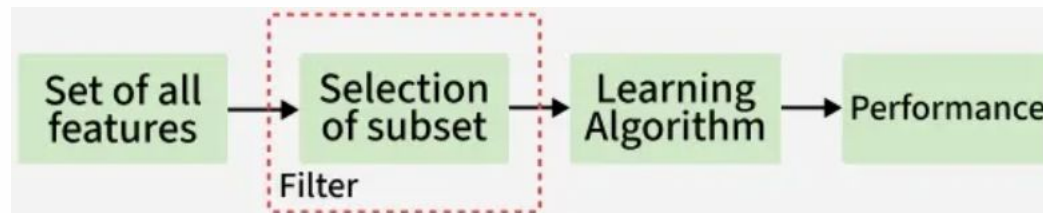
Importance of Feature Selection in Machine Learning

- 1. Enhanced Model Performance:** Selecting relevant features leads to more accurate and efficient models.
- 2. Reduced Overfitting:** Including irrelevant features can lead to overfitting, where the model performs well on the training data but poorly on test data. Feature selection helps mitigate this issue.
- 3. Improved Interpretability:** Models with fewer features are easier to interpret and explain to stakeholders.
- 4. Faster Training and Inference:** Working with a reduced set of features speeds up both the training and inference phases of the machine learning pipeline.

Types of Feature Selection Techniques:

1. Filter Methods

Filter methods assess the relevance of features based on statistical measures based on their correlation with the target variable. Examples include chi-square test, information gain, and correlation coefficient. Features are selected based on their individual characteristics, without considering the machine learning algorithm used.



For example:

```
from sklearn.feature_selection import SelectKBest, f_classif
# Example of SelectKBest with ANOVA F-value for classification
selector = SelectKBest(score_func=f_classif, k=5)
X_selected = selector.fit_transform(X, y)
```

Advantages

Fast and efficient: Filter methods are computationally inexpensive, making them ideal for large datasets.

Easy to implement: These methods are often built-in to popular machine learning libraries, requiring minimal coding effort.

Model Independence: Filter methods can be used with any type of machine learning model, making them versatile tools.

Limitations

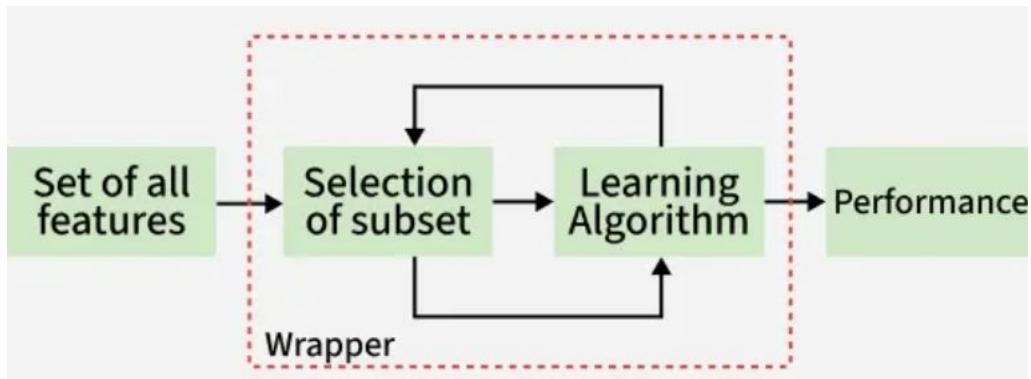
Limited interaction with the model: Since they operate independently, filter methods might miss data interactions that could be important for prediction.

Choosing the right metric: Selecting the appropriate metric for our data and task is important for optimal performance.

Types of Feature Selection Techniques:

2. Wrapper Methods

Wrapper methods use a specific machine learning algorithm to evaluate feature importance. They select features based on their impact on model performance. They wrap the feature selection process around the model training and evaluate the model's performance to determine the optimal subset of features. Wrapper method follows a greedy search approach by evaluating all the possible combinations of features against the evaluation criterion. Examples include recursive feature elimination (RFE) i.e. eliminating the feature having lowest coefficient and forward/backward stepwise selection.



For example:

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

# Recursive Feature Elimination (RFE) with Logistic Regression
estimator = LogisticRegression()

selector = RFE(estimator, n_features_to_select=5)
X_selected = selector.fit_transform(X, y)
```

Techniques

Forward Selection: Start with no features and add one at a time based on improvement.

Backward Elimination: Start with all features and remove the least useful ones.

Recursive Feature Elimination (RFE): Removes the least important features step by step.

Advantages

Model-specific optimization: Wrapper methods directly consider how features influence the model, potentially leading to better performance compared to filter methods.

Flexible: These methods can be adapted to various model types and evaluation metrics.

Limitations

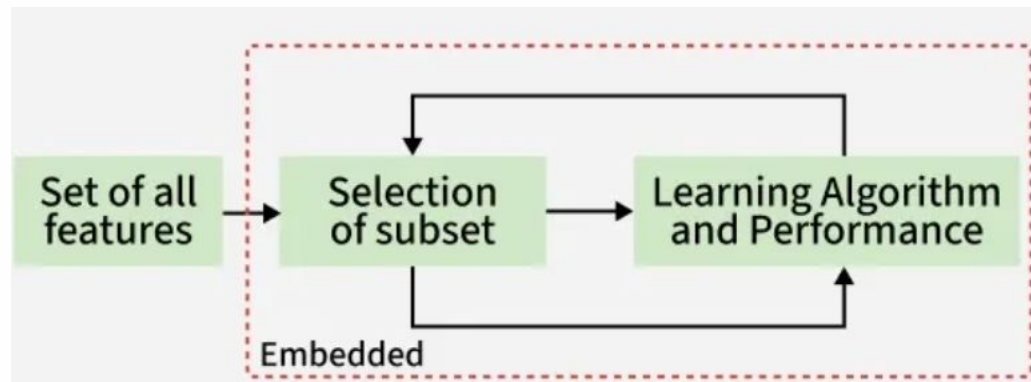
Computationally expensive: Evaluating different feature combinations can be time-consuming, especially for large datasets.

Risk of overfitting: Fine-tuning features to a specific model can lead to an overfitted model that performs poorly on unseen data.

Types of Feature Selection Techniques:

3. Embedded Methods

Embedded methods incorporate feature selection into the model training process. They select features based on their contribution to the model's predictive power. The feature selection is embedded within the algorithm, and the model selects the most relevant features during training. Examples include LASSO (Least Absolute Shrinkage and Selection Operator) i.e. adding a penalty to the different parameters of the machine learning model to reduce the freedom of the model and tree-based methods like Random Forests, which assign feature importance scores.



```
from sklearn.feature_selection import SelectFromModel
from sklearn.ensemble import RandomForestClassifier
```

```
# SelectFromModel with Random Forest Classifier
selector =
```

```
SelectFromModel(RandomForestClassifier(n_estimators=100))
X_selected = selector.fit_transform(X, y)
```

Techniques

- L1 Regularization (Lasso): Keeps only features with non-zero coefficients.
- Decision Trees and Random Forests: Select features based on impurity reduction.
- Gradient Boosting: Pick features that reduce prediction error the most

Advantages

- Efficient and effective: Embedded methods can achieve good results without the computational burden of some wrapper methods.
- Model-specific learning: Similar to wrapper methods these techniques uses the learning process to identify relevant features.

Limitations

- Limited interpretability: Embedded methods can be more challenging to interpret compared to filter methods making it harder to understand why specific features were chosen.
- Not universally applicable: Not all machine learning algorithms support embedded feature selection techniques.

Types of Feature Selection Techniques:

3. Hybrid Methods

Hybrid methods combine aspects of filter, wrapper, and embedded methods. They leverage the strengths of multiple techniques. Dimensionality reduction techniques, such as Principal Component Analysis (PCA) transform the original features into a lower-dimensional space while preserving important information. These methods are effective for reducing the feature space but do not explicitly select individual features.

For example:

```
from sklearn.feature_selection import SelectFromModel
```

```
from sklearn.linear_model import LogisticRegression
```

```
# Hybrid method combining SelectFromModel and Logistic Regression
```

```
selector = SelectFromModel(LogisticRegression(penalty="l1", C=0.1))
```

```
X_selected = selector.fit_transform(X, y)
```

Applicability of Feature Selection Technique:

Small Datasets: Feature selection is beneficial for datasets with a large number of features relative to the number of samples.

High-Dimensional Data: In cases where the dimensionality of the data is high, feature selection helps in reducing computational complexity and improving model performance.

Interpretability: When interpretability is crucial, selecting a subset of features facilitates easier model understanding and explanation.